The Project Graveyard

Unfinished isn’t failed. Lay old projects to rest, understand why they stopped, and decide whether they deserve a second life.

Code Variant · Build 01 · Final submission · **Problem Statement 05**

Rajnish Kumar Verma (solo)

Live: project-graveyard-codevariant.vercel.app Code: github.com/erajneeshverma/rajnishkver-CodeVarient

What we shipped in 12 hours



6 screens, all drag-and-drop

31 automated tests passing

0 accessibility violations (axe-core)

30+ commits, each one deployed live

A working, deployed product: bury a project, run its autopsy, get explained advice, decide its fate. Then the hour-6 twist turned the whole app into a drag-and-drop canvas.

Unfinished projects don’t die, they get forgotten

Context decays

Months later you can’t remember what was done, what was left, or why you stopped.

No honest post-mortem

Projects stall for very different reasons (time, scope, a blocker, lost interest), but nobody records which one.

Only two choices today

“Ignore it” or “delete it”. Restarting, pivoting or salvaging parts is never seriously considered.

Our reading of the problem

Storage isn’t the problem. GitHub already keeps the code.

The missing piece is a decision: *what is this project worth now, and what should I do with it?*

Bury it in under two minutes

1

The Tombstone

What it was, the goal, what got done and what didn't. Paste a GitHub URL and the stack, dates and README fill themselves in.

2

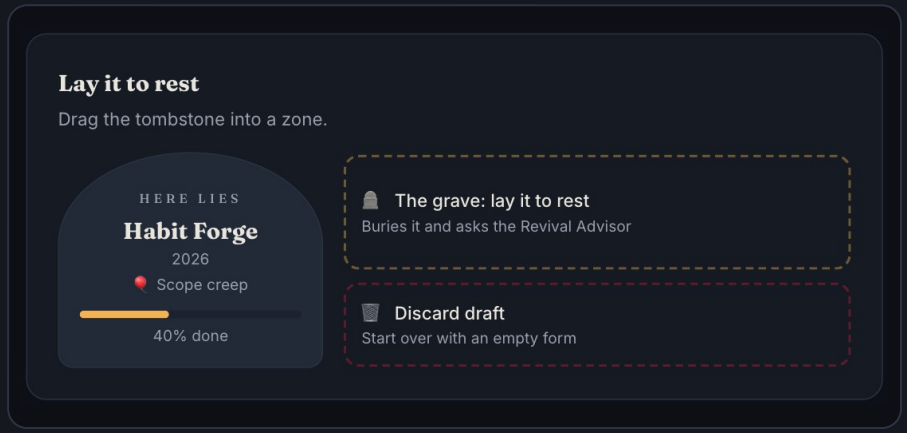
The Autopsy

Why it stopped: time, scope creep, tech blocker, tech debt, lost interest, no users, plus an honest note.

3

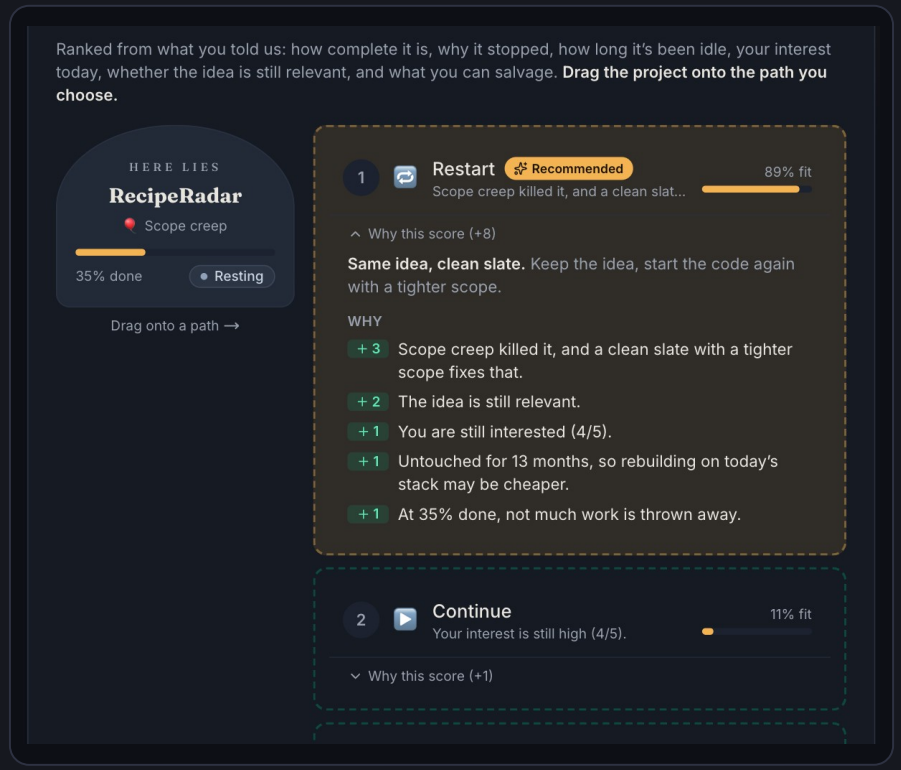
Assess

Interest today (1–5) and is it still relevant? The advisor previews its pick before you finish.



Finish by dragging the live tombstone into **the grave**. Progress autosaves; discarding a draft can be undone.

The Revival Advisor shows its working



Six signals

- Completion %
- Cause of death
- Months idle
- Interest today
- Still relevant?
- Salvageable parts

Five paths
Continue · Restart · Change the idea · Repurpose · Archive

+3 Scope creep killed it, and a clean slate with a tighter scope fixes that.

Rule-based on purpose: every point has its sentence, fit % shares one scale so it always agrees with the ranking, and the rules live in one config file. You always make the final call, and overriding is logged.

HOUR 6 · 18:00

The Canvas Protocol

“No standard buttons for primary actions. Users must drag data cards into Action Zones.”

First drag-and-drop screen shipped 16 minutes later. Every primary action converted by 18:28.

Every button became a drag

Action	Before	Now: drag … into …
Bury a project	“Lay it to rest” button	the live tombstone → The grave
Decide its fate	“Choose” + modal	the project card → a ranked path zone
Revive or archive fast	open project, choose	a tombstone → Resurrection Circle / Eternal Rest
Record a revisit	“Update advice”	the assessment card → Record revisit
Haunt · Export · Delete	buttons + confirm	the project → Haunt / Post-mortem / Exhume
Salvage · Timeline	“Add” buttons	the new part / entry card → its list
Backup · Import	file buttons	the graveyard card → Export · a .json file → Import

Everything destructive can be undone from its toast. The logic didn’t change, only the way you trigger it.

The home screen is now a canvas



Séance
ask the advisor

Resurrection Circle
revive with the best path

Eternal Rest
archive, R.I.P.

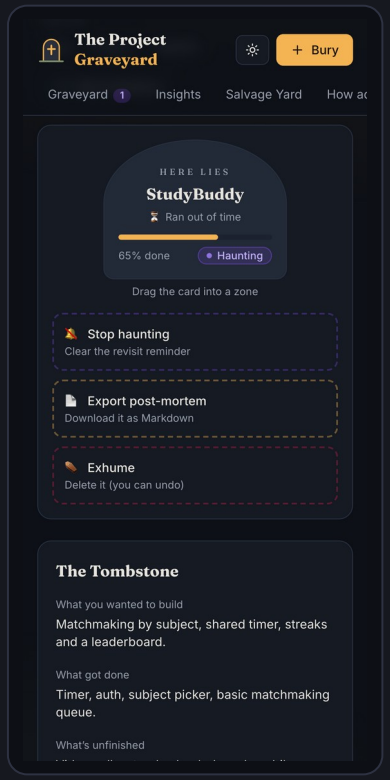
Haunting Ground
remind me in 30 days

Post-mortem
download its report

Exhume
delete, with undo

Tombstones stand in a field you arrange yourself; drop one anywhere to move it. Every drop plays an effect: light for a revival, dust for a burial, a ghost for a haunt.

Drag and drop that works for everyone



Keyboard

Space picks a card up, arrow keys jump between zones (off-screen ones scroll into view), Space drops, Esc cancels.

Screen readers

Every pick-up, hover and drop is announced. Cards are named by what they show.

Touch

Long-press to lift on phones and tablets; scrolling still works. Dropping on empty space does nothing.

Safe by default

Undo on every destructive drop, plus zero axe-core violations on every page, light and dark.

Beyond the brief

Graveyard Insights

Top cause of death, revival rate, plain findings.

Post-mortem export

Any project as a Markdown report for its repo.

Salvage Yard

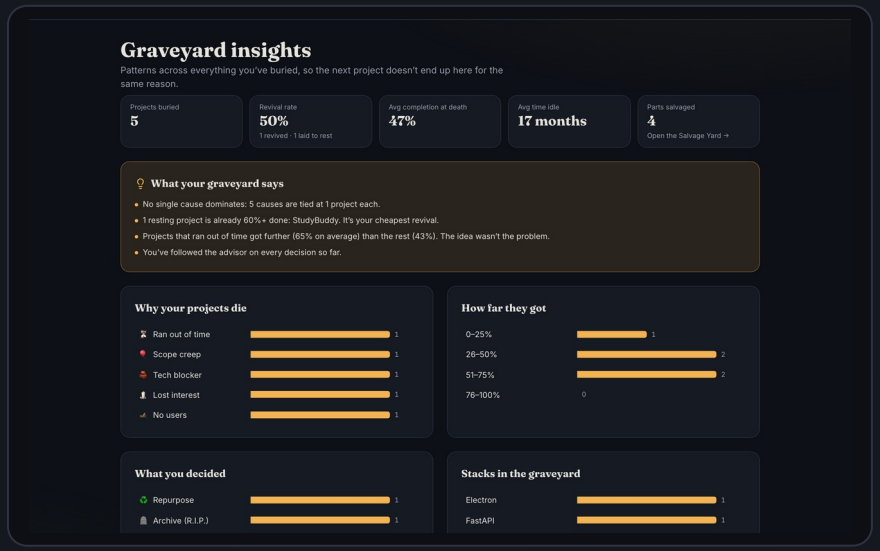
Reusable code, lessons and assets from every project.

Haunt me later

Revisit reminders; due projects start haunting you.

GitHub import

Paste a public repo: stack, dates and README fill in.



Plus light and dark themes, JSON backup and restore, and a full timeline for every project.

One canvas layer, zero backend

Config
causes, statuses, paths, every rule

›

Revival engine
pure TypeScript scoring, tested

›

Store
Zustand, validated, persisted locally

›

Canvas layer
DataCard + ActionZone (dnd-kit)

›

Screens
Canvas · Bury · Project · Insights

React 19 + Vite | TypeScript | Tailwind CSS v4 | dnd-kit | Zustand | Framer Motion | Vitest | Vercel

Why the twist took minutes, not hours

Every screen reuses the same two canvas components, so mouse, touch, keyboard and screen-reader support were written once. The engine, store and rules were untouched: the twist changed how actions are triggered, not what they do. Data is validated on import and on load, so a bad file can never blank the app.

How the build went



Every checkpoint was submitted with the same repo. Every commit deployed itself to the live site in about 20 seconds, and every change was checked by tests in a real browser.



Every abandoned project deserves a second look

Understand it, decide its fate, and give the good ideas another chance. And when the rules change, evolve instead of starting over.

Try it live **project-graveyard-codevariant.vercel.app**

Source code github.com/erajneeshverma/rajnishkver-CodeVarient

Thank you · Rajnish Kumar Verma